

High Tech Systems Honors

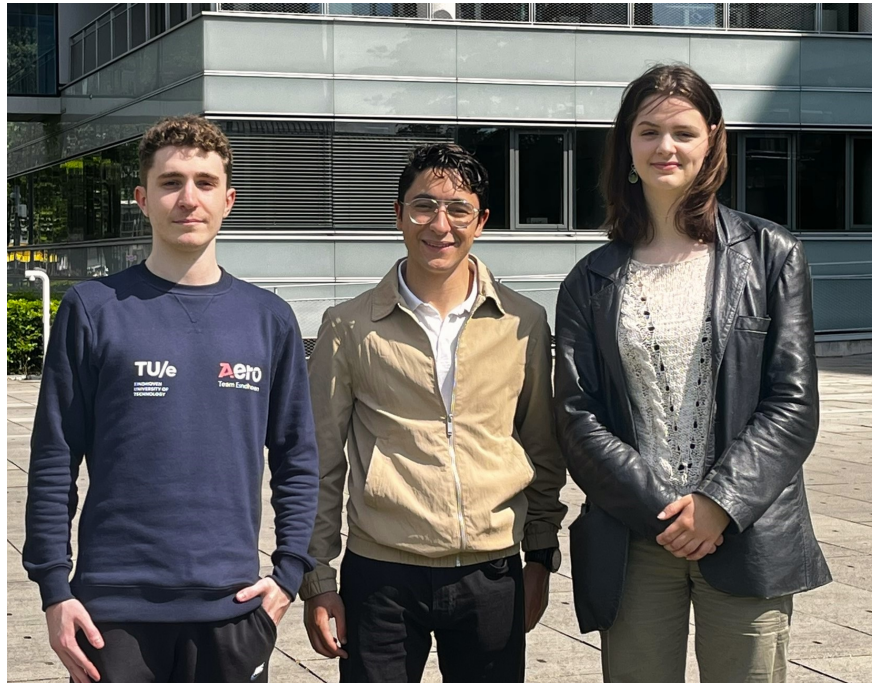


SymAware Honors Report

31-09-2024 to 25-05-2025

Full Name	Student ID	Study
Nora Baljé	1839888	Computer Science
Daniel Tyukov	1819283	Electrical Engineering
Ismail Hassaballa	1569457	Electrical Engineering

Coached by Sofie Haesaert
Special thanks to Ruohan Wang



Eindhoven, May 18, 2025

Contents

1	Introduction	2
1.1	Problem statement	2
1.2	Literary Research	3
1.2.1	V2D	3
1.2.2	V2P	3
1.2.3	V2V	3
2	Process	4
2.1	LIDAR Data Processing for Risk Field Calculation	4
2.1.1	Introduction	4
2.1.2	ROS Message Format	5
2.1.3	ROS Implementation	5
2.1.4	Preliminary: Octree Structure	6
2.1.5	Nearest Object Detection	7
2.1.6	Setup	7
2.2	Risk assesement	9
3	Results	11
3.1	LIDAR Data Processing for Risk Field Calculation	11
3.1.1	Testing	11
3.1.2	Challenges	12
3.2	risk field results	12
4	Next year	13
4.1	LIDAR Future Work	13
4.1.1	Distance-Dependent Feature Extraction	13
4.1.2	Enhanced Clustering and Segmentation Techniques	13
4.2	Expand our project	14
5	Use of AI tools	15
	References	16
A	Individual contributions	17
A.1	Nora	17
A.2	Ismail	18
A.3	Daniel	19
B	Code	20
B.1	Final Risk Field Simulation Code	20

1 Introduction

1.1 Problem statement

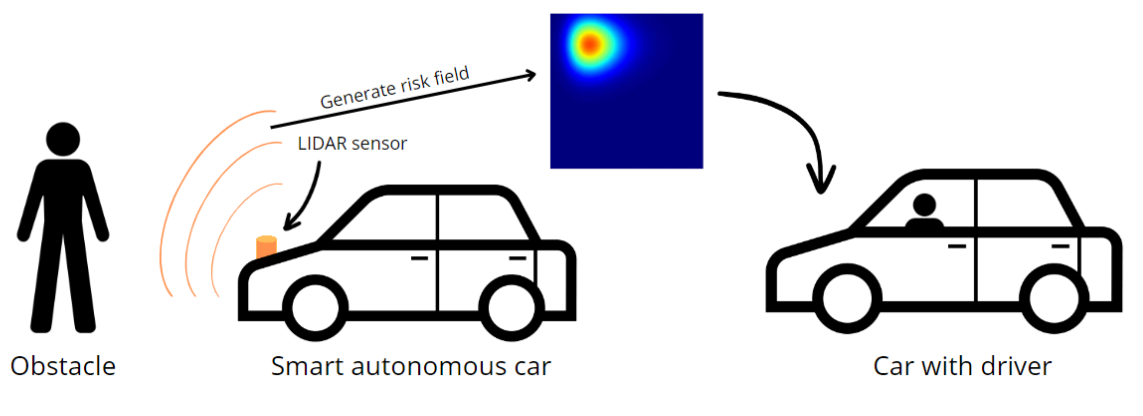
Imagine driving on the road when suddenly the car in front of you makes an unexpected emergency stop. Now you have to abruptly stop, hoping to not crash into the vehicle in front of you. This abrupt stop will cause all the cars behind you to also brake sharply, which can lead to a very dangerous chain collision. These situations can cause a lot of injuries and potentially multiple fatalities.

Globally we have already been doing a lot to increase road safety, and we have made significant improvements. However, removing all dangerous situations will never happens as sometimes danger just appears out of the blue. So if removing the danger is not possible, how can we ensure that as little damage as possible is done when facing danger? For this we wanted to find a way to increase road awareness. If a driver is more aware of danger they can react earlier and avoid damages.

Lately, with the focus on autonomous vehicle (AV) technology, several efforts were made in enhancing safety and communication between AVs. The main focus was mainly on AV-to-AV interactions leading to advanced systems capable of managing complex driving scenarios autonomously. This concentrated effort has, however, led to a neglect of the equally important interaction between autonomous vehicles and traditional vehicles (TVs).

Traditional Vehicles still form the majority of road traffic but lack the advanced sensors and communication capabilities of AVs. This advanced technology enables AVs to detect obstacles and risks that a driver of a traditional vehicle might miss. We hope to enable AVs to increase the awareness of TVs and subsequently increase road safety. Challenge however would be, how to determine if an obstacle actually poses a potential risk, how can we quantify it and finally, how to transmit and communicate it with the TV.

The diagram presented below showcases the structure that we where hoping to create.



1.2 Literary Research

Before we were able to jump into the practical part of our project we spend some time doing literary research. This was for us to understand the world of AVs but also for us to formulate a concrete idea. The research was conducted in 3 areas, vehicle-to-driver communication (V2D), vehicle-to-pedestrian communication (V2P) and vehicle-to-vehicle (V2V) communication.

1.2.1 V2D

When reading about V2D interaction we quickly realized that this area did not interest us. There are a lot of different levels of automation that have different requirements [8], this leads to products that are only applicable to a very small target audience. The potential for V2D communication is significantly smaller than the other two options, so we decided against this path.

1.2.2 V2P

When reading about V2P interaction we were initially very intrigued. We noticed that a lot of work had already been done and significant progress had been made. We however, were looking for an area that had a concrete issue we could work on. For V2P communication the main issue appears to us as education. When looking at a study [2] that presents a tool to communicate between AVs and pedestrians we can clearly see that the tool only helps pedestrians when the behaviour of the car matches the signal. This indicates that the tool works fine but people do not yet trust the signals the AV gives. As we wanted to focus on developing something we did not think this research area had a lot of potential for us.

1.2.3 V2V

In this area there are a lot of possibilities. V2V communication is generally seen as being in a good state. However, there are possible improvements when it comes to communication with regular vehicles. We want to utilize this missed opportunity and create an interface that communicate risk to traditional vehicles. Specifically, the idea of a shared risk field [4] really interested us and this is what we based our problem statement upon.

2 Process

2.1 LIDAR Data Processing for Risk Field Calculation

2.1.1 Introduction

The LIDAR system is one of the most important inclusions for the new era vehicle. LIDAR systems play a big part in giving high-resolution, real-time data on surroundings that is used for navigation and collision avoidance. The LIDAR sensors send laser pulses, and then the time taken by each pulse to return after striking an object is measured. This results in a very detailed 3D map of the environment, important for situational awareness. [7].

The project uses the Hokuyo UST-10LX LIDAR sensor because its specifications are very high: 40 Hz scanning frequency, a maximum range of 10 m, and a good angular resolution of 0.25 degrees. Such characteristics enable the sensor to detect and track objects in dynamic environments. The LIDAR scans the environment radially: it perceives a 270° field of view with a resolution of 0.25° , and thus, it obtains 1081 data points on every scanning step.

The primary measurements from the LIDAR are:

- **Range (r_i):** The distance from the LIDAR to the nearest object in each scanned direction.
- **Angle (θ_i):** The angle at which each distance measurement is taken, relative to the LIDAR's reference direction.

The LIDAR sensor Hokuyo UST-10LX is shown in Figure 1. The principal component of the approach will be to transform the LIDAR data acquired from a polar grid into a Cartesian grid that should adequately model the surrounding environment.

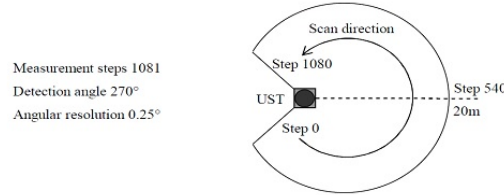


Figure 1: Hokuyo UST-10LX LIDAR sensor.

Object detection and the calculations of the risk field is based on a threat field model from LIDAR data, elaborately described in reference paper [6] and [1]. This model interprets a spatiotemporally varying scalar field representing collision potentials on regions around the vehicle. Therefore, the addition of LIDAR data to that model provides a risk assessment to a required accuracy.

The mathematical transformation from polar coordinates (r, θ) to Cartesian coordinates (x, y) is given by:

$$x_i = r_i \cos(\theta_i)$$

$$y_i = r_i \sin(\theta_i)$$

where r_i is the range at angle θ_i . This transformation is crucial for integrating LIDAR data into the threat field model. Figure 2 [5] shows the transformation process from polar to Cartesian coordinates.

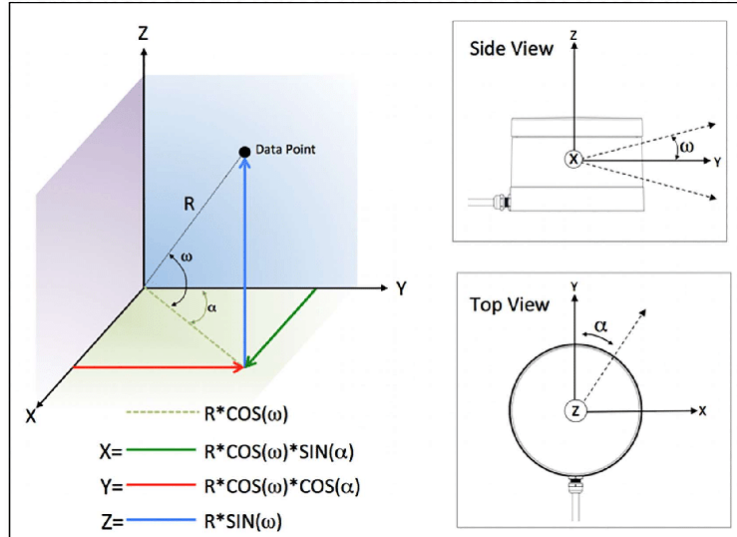


Figure 2: Transformation from polar to Cartesian coordinates.

2.1.2 ROS Message Format

In ROS, LIDAR data is published on the `/scan` topic in the `sensor_msgs/LaserScan` message format. The `LaserScan` message consists of a few fields; most important for our application are:

- **ranges** - An array of float32 ranges of the closest object for each angle increment in front of the robot.
- **intensities**: Array of float32, corresponding intensities of the reflected laser pulse at each angle increment.

The structure of the `LaserScan` message is as follows:

```
Header header
float32 angle_min
float32 angle_max
float32 angle_increment
float32 time_increment
float32 scan_time
float32 range_min
float32 range_max
float32[] ranges
float32[] intensities
```

2.1.3 ROS Implementation

Executing in ROS involves the subscription to the `/scan` topic, processing of incoming `LaserScan` messages, and conversion to Cartesian coordinate from polar. Following is the node in Python for ROS-based execution:

```
#!/usr/bin/env python
import rospy
from sensor_msgs.msg import LaserScan
import math

def polar_to_cartesian(ranges, angle_min, angle_increment):
    points = []
    for i, r in enumerate(ranges):
        if r > 0:
            angle = angle_min + i * angle_increment
```

```

        x = r * math.cos(angle)
        y = r * math.sin(angle)
        points.append((x, y))
    return points

def scan_callback(msg):
    points = polar_to_cartesian(msg.ranges, msg.angle_min, msg.angle_increment)
    rospy.loginfo("Cartesian Points: %s", points)

def listener():
    rospy.init_node('lidar_listener', anonymous=True)
    rospy.Subscriber("/scan", LaserScan, scan_callback)
    rospy.spin()

if __name__ == '__main__':
    listener()

```

The node subscribes to the `/scan` topic, reads in LIDAR data, converts each range and angle pair into Cartesian coordinates, and logs the resulting points for further processing.

2.1.4 Preliminary: Octree Structure

We have taken the case of a hierarchical arrangement of data, called the Octree structure. We have experimented in the laboratory towards an efficient system to represent 3D point clouds, helping work with inherently unstructured and sparse LIDAR point clouds. At the time of forming the Octree, the entire point cloud is taken as a cube or a voxel and then subdivided into 8 sizes of equal sub-voxels. Here the process is repeated until the voxel side length of a leaf equals the quantization step. The operation of quantization merges the points into the same leaf voxel. This bit 1 represents the sub-voxels that contain points and the empty ones are represented by a value of 0 for each internal voxel. Thus, the occupancy symbols for each of the internal voxels are composed of 8 bits (1 to 255), where each of the bits states the occupation status of the corresponding sub-voxel [6].

Spherical coordinates are inherently better suited for representing the radial nature of LIDAR data, as they naturally align with the way LIDAR sensors capture the environment. The complexity of data is reduced and it becomes easy to bring out the simplification of circular and spherical objects. This alignment helps in easily representing circular and spherical objects such that the data is not complex. It further evolves into an octree structure that allows more efficient hierarchical representation for 3D point clouds with more storage and retrieval efficiency. This kind of hierarchical approach also allows solving adaptation: due to distance from objects, these could be represented at a less detailed rate, in order to ease the computational load without affecting very much the accuracy towards the nearby objects.

The radial nature of LIDAR data is naturally best illustrated in spherical coordinates. Alignment further eases the way for the representation of circular and spherical objects; therefore, it aids in easing the complexity of the data. Octree provides an efficient hierarchical structure in which adaptive resolution may further enhance efficiency during storage and retrieval for modeling 3D point clouds. In such a type of hierarchical approach, adaptive resolution can be introduced: Far away objects can be represented at low resolution hence decrease the computational load without losing the accuracy of nearer objects. We use different coordinate systems, 3-D Cartesian, Cylindrical, and Spherical coordinates.

$$\rho = \sqrt{x^2 + y^2}, \quad \theta = \arctan\left(\frac{y}{x}\right), \quad z = z$$

$$\rho = \sqrt{x^2 + y^2 + z^2}, \quad \theta = \arctan\left(\frac{y}{x}\right), \quad \phi = \arccos\left(\frac{z}{\rho}\right)$$

For spherical coordinate-based learned point cloud compression, we use:

$$q_\theta = \frac{2\pi}{b-1} \approx \frac{2\pi q}{\rho_{\max}}$$

$$q_\phi = \frac{\pi}{b-1} \approx \frac{\pi q}{\rho_{\max}}$$

2.1.5 Nearest Object Detection

Detecting the nearest object is crucial for real-time risk assessment and collision avoidance in autonomous vehicles. The following algorithm outlines the steps for detecting the nearest object using LIDAR data:

Nearest-object detection allows obtaining a real-time risk assessment for making collisions in advanced driving assistance. The following algorithm will give the steps of detecting the nearest object within collected LIDAR data, in our current implementation:

```
def find_nearest_object(ranges):
    min_distance = float('inf')
    min_index = -1
    for i, distance in enumerate(ranges):
        if 0 < distance < min_distance:
            min_distance = distance
            min_index = i
    return min_index, min_distance
```

The looping of the algorithm happens on the range data of LIDAR to find the minimum distance. This distance will point to the nearest object, following which it converts the polar coordinates into Cartesian for better processing.

2.1.6 Setup

Setup The Hokuyo UST-10LX LIDAR sensor was mounted at the front of the vehicle, and the setup of the ROS environment to read and process data from the LIDAR generally followed these steps:

1. **Mounting the LIDAR Sensor:** Secure the Hokuyo UST-10LX LIDAR sensor using a rigid mounting bracket to the front of the vehicle.
2. **Connecting to ROS:** Connect the LIDAR sensor to the NVIDIA Jetson onboard computer, ensuring that the ROS environment is set up in a way where it can recognize the LIDAR sensor.
3. **Start ROS Nodes:** Start the ROS node which acquires the LIDAR data using the command:

```
roslaunch hokuyo_node hokuyo_test.launch
```

4. **Verify Data Acquisition:** Use the ROS topic echo command to verify that LIDAR data is being published correctly:

```
rostopic echo /scan
```



Figure 3: Setup of the Hokuyo UST-10LX LIDAR sensor on the vehicle.

To achieve the goal mentioned in the problem statement we needed to first achieve some practical knowledge. We can split the problem statement into 3 separate sections:

1. Vehicle one can assess risk
2. Vehicle one can communicate said risk to vehicle two
3. Vehicle two can communicate said risk to the driver

Of course all of this needs to happen in real time which provides an additional challenge. As the interface would be used in highly dangerous situations we cannot afford to make any mistakes.

A very challenging assignment thus for 3 inexperienced bachelor students. This is why we started with focusing on one task and getting the necessary knowledge required to achieve this one. However, no matter what step we chose to start with, we needed to understand how the RC-car worked, so this was issue number one. Luckily as we were building upon SymAware, they were able to provide us with a step by step tutorial for the RC-cars. Though we did struggle a lot when discovering the car through these tutorials we managed to complete it, so we were ready for the 'real deal'.

We chose to tackle task one, risk-assessment by a vehicle, at first. We hoped to achieve this by using PreScan, a software created by Siemens in which you can create virtual realistic road situations. Unfortunately this software proved to be quite difficult, and the integration between PreScan and ROS, another software we are using, was still in the developmental phase. Thus to still be able to generate a result we opted for using the LIDAR sensor instead. This is a sensor that was already present on the vehicle that can see up to 10 meter ahead and has 270 degree vision. Using the LIDAR sensor did mean that we had to physically put an obstacle in front of the vehicle, this made the process more prone to human errors which was a bit difficult.

After figuring out the LIDAR sensor we had the data, but we had to find a way to quantify and visualize the data. This is where our literary research came in, as one of the papers we read provided us with the exact mathematics we needed to quantify the risk correctly. Understanding the math was quite hard but

once we did we were able to create a python program that displayed the risk field. After combining these two findings, the LIDAR data and the calculation of the risk field, we were able to display the risk field in real time. This was amazing, but we quickly ran into an issue it took the program 2 seconds to calculate one frame of the risk field. We need this data to be updating in real time and in 2 seconds the entire road situation could be changed. So this was not fast enough at all, we needed to find a way to make it faster.

We tried a lot of different things but none of them appeared to make a significant difference. Until we converted our program to a python program that can be compiled with Numba. This decreased our calculation time from, 2 seconds to 0.03. This change made it possible for our program to assess in real time and thus we have achieved the first task.

As we spend the majority of this year learning the skills necessary we did not have enough time to achieve the next two tasks but we hope to expand on this project next year and complete our idea.

2.2 Risk assessment

A big challenge for our project was quantifying and measuring the risk posed by other objects/ obstacles that the AV and TV could face. Our final risk assessment, greatly inspired from the research paper, quantifies risk and composes a scalar risk field of a 2d map. The risk calculations take into account:

1. The speed of current vehicle.
2. The speed of the other obstacles.
3. The distance between the vehicle and other obstacle.
4. The size of the obstacle.
5. The safety distance between two cars.

Collision Risk Field Math

Important Definitions

$$\begin{aligned}\mathbf{p} &= (x, y) \quad \text{position of TV} \\ \mathbf{p}_i &= (x_i(t), y_i(t)) \quad \text{position of } i\text{-th vehicle}\end{aligned}$$

Collision Risk Function

The collision risk function should consider the velocities of vehicles, relative position to TV, and the "three second" safety rule separation distance.

Let v_0 be the TV speed:

$d_{ix} = 3(v_0 + |v_{ix}(t)|)$ where 3 sets the "three second" safety separation rule between the TV and other vehicle i

$$\begin{aligned}\beta_{ix} &= \ln \left(\frac{d_{ix} \cdot (v_0 - |v_{ix}(t)| - \varepsilon_2)}{2(|v_{ix}(t)| + \varepsilon_2)} \right) \\ \beta_{iy} &= \ln \left(\frac{d_{iy} \cdot (\varepsilon_3 - |v_{iy}(t)| + \varepsilon_2)}{2(|v_{iy}(t)| + \varepsilon_4)} \right)\end{aligned}$$

where

$$\varepsilon_1 = \frac{1}{\sqrt{-\ln(\varepsilon_0^2)}}$$

and $\varepsilon_0, \varepsilon_2, \varepsilon_3, \varepsilon_4$

are user-defined parameters and ε_0 is the threshold for considering risk $0 < \varepsilon_0 < 1$

$$\alpha_{ix} = \varepsilon_1 \ln \left(\frac{v_0 + |v_{ix}(t)| + \varepsilon_2}{v_0 - |v_{ix}(t)| - \varepsilon_2} \right)$$

$$\alpha_{iy} = \varepsilon_1 \ln \left(\frac{\varepsilon_3 + |v_{iy}(t)| + \varepsilon_4}{\varepsilon_3 - |v_{iy}(t)| - \varepsilon_4} \right)$$

Additionally, taking into account the TV position:

$$\delta_{ix}(\mathbf{p}, t) = (\text{sgn } v_{ix}(t))(x - p_i(t)) + e^{\beta_{ix}}$$

$$\delta_{iy}(\mathbf{p}, t) = (\text{sgn } v_{iy}(t))(y - p_{iy}(t)) + e^{\beta_{iy}}$$

Finally, the collision threat for the vehicle is defined while considering a lognormal function:

$$f_i = e^{-\frac{1}{2} \left(\frac{\ln(\delta_{ix} - \beta_{ix})}{\alpha_{ix}} \right)^2} e^{-\frac{1}{2} \left(\frac{\ln(\delta_{iy} - \beta_{iy})}{\alpha_{iy}} \right)^2} \quad \text{only for } \delta_{iy}(\mathbf{p}, t) > 0 \wedge \delta_{ix}(\mathbf{p}, t) > 0$$

otherwise $f_i = \varepsilon_5$

Hence, the scalar risk field will be a summation of all collision threats for N other vehicles:

$$W(\mathbf{p}, t) = \varepsilon_6 \sum_{i=1}^N C_i(\mathbf{p}, t)$$

3 Results

3.1 LIDAR Data Processing for Risk Field Calculation

3.1.1 Testing

1. **Static Obstacle Detection:** Place a stationary object at different distances and positions relative to the view of the LIDAR in both lateral and frontal environments. Check if the algorithm correctly detects the closest object and gives the correct distance.
2. **Dynamic Obstacle Detection:** Make objects in the LIDAR's field of view move to simulate real-world dynamic cases. Validate that your algorithm can monitor and update real-time changes with respect to the nearest object from its position.
3. **Edge Cases:** Test the robustness of the algorithm for edge cases, like when a surface is partly $\lambda/4$ reflective or targets overlap, and observe the behavior of a system toward the the said scenario.

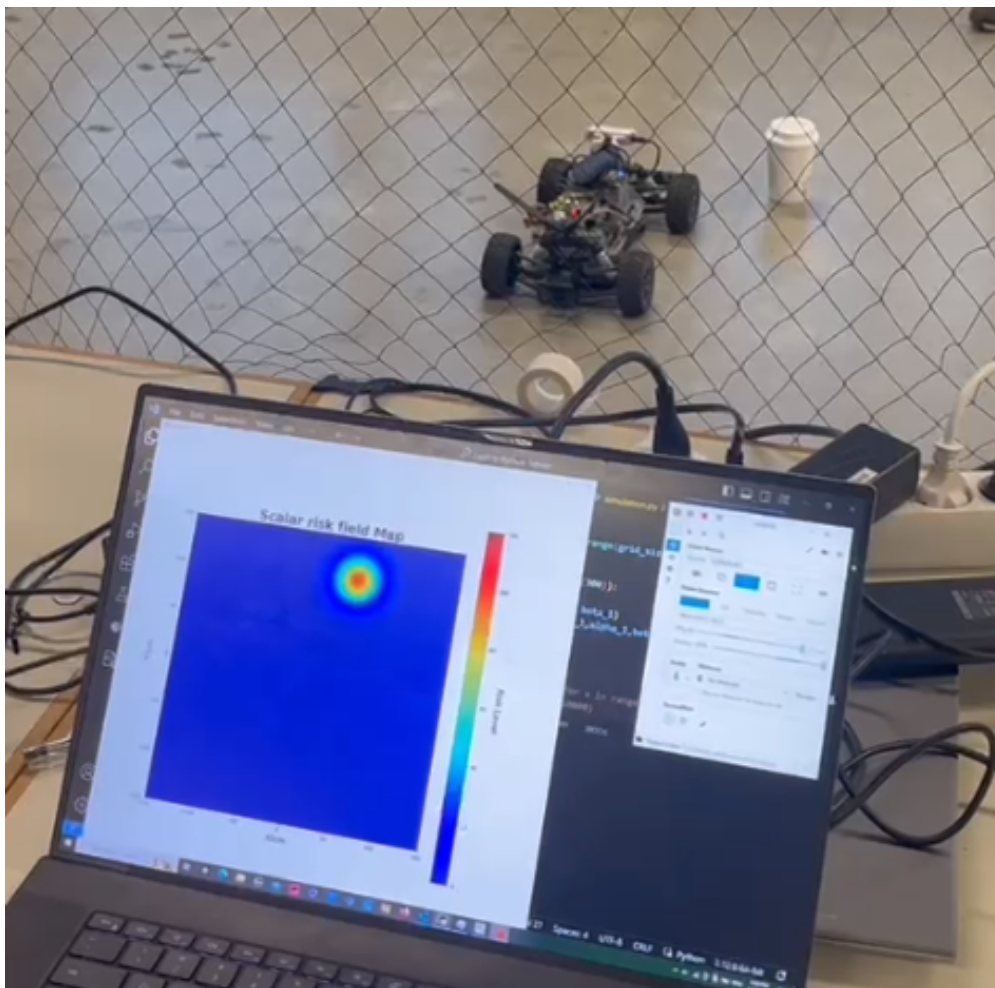


Figure 4: Testing scenarios for validating the nearest object detection algorithm.

3.1.2 Challenges

We encountered many areas of challenge where innovative solutions were needed during the implementation:

1. **Sensor Data Noise:** Initial tests indicated that the LIDAR data was very noisy and thus, in turn, caused the object detection to have poor accuracy. We used a median filter to smooth and reduce noise for the data.
2. **Shiny Surfaces:** Some very shiny surfaces resulted in output errors for the distance measurement. Due to this, we had to calibrate the intensity threshold of the LIDAR to remove readings that were considered unreliable.
3. **Computational Load:** Real-time processing of LIDAR data is extremely high in computational load. We have optimized the same by implementation of the algorithm using efficient data structures and parallel processing techniques.

These challenges gave valuable learnings and enabled us to fine-tune our system to obtain better performance in real-time scenarios.

3.2 risk field results

The project initially required additional to fine-tune the model and adjust the parameters. We discovered that the risk quantification model needed significant tweaking to accurately reflect real-world conditions. This involved a thorough process of parameter adjustment, ensuring that the model was both accurate and reliable.

One of our key improvements was enhancing the visualization of the risk field. This improvement was crucial for interpreting the results and demonstrating the effectiveness of the model. Additionally, we focused on increasing the calculation efficiency, allowing for faster processing and real-time analysis.

After completing these adjustments, the refined model provided reliable risk assessments, and the improved visualization techniques made it easier to understand and communicate the findings.

4 Next year

4.1 LIDAR Future Work

4.1.1 Distance-Dependent Feature Extraction

We intend to extend the approach of near object identification. [3] and include advanced distance-dependent feature-extraction techniques. The most important problem related to LIDAR data is the changing density and point spatial arrangement when objects move further from the sensor. This variability influences the performance of CNNs that require a consistent feature representation across the input space.

For this, we design the multiscale network in a way that the level of feature extraction is conditioned based on the distance at which an object is detected. The distance-based filters we use are optimized with respect to the distance range chosen for a LIDAR sensor. The mathematical formulation for the network is given by:

$$F_i(x) = \begin{cases} f_1(x) & \text{if } d(x) \leq d_1 \\ f_2(x) & \text{if } d_1 < d(x) \leq d_2 \\ \vdots & \\ f_n(x) & \text{if } d_{n-1} < d(x) \end{cases}$$

where $F_i(x)$ represents the feature extraction function applied to point x based on its distance $d(x)$, and $f_1, f_2, \dots, f_n$ are the feature extraction functions optimized for different distance ranges $d_1, d_2, \dots, d_{n-1}$.

4.1.2 Enhanced Clustering and Segmentation Techniques

We also plan to incorporate improved clustering and segmentation algorithms to better handle the segmentation of ground and non-ground points, building on the methods described by Zhao et al. [7]. By leveraging kd-tree data structures, we can efficiently manage the large number of sparse 3D points in LiDAR datasets, enabling more accurate and faster nearest neighbor searches.

The complexity analysis of kd-tree operations, as detailed in [7], provides a solid foundation:

We shall obtain the better integration of the reformed algorithms for clustering and segmentation by addressing the segmenting of ground and non-ground points, with the enhancements proposed in work by Zhao et al. [7]. The sparse 3D LiDAR datasets described include a large amount of data that can be effectively managed using the kd-tree data structure, thus guaranteeing speed in searches for the nearest neighbors.

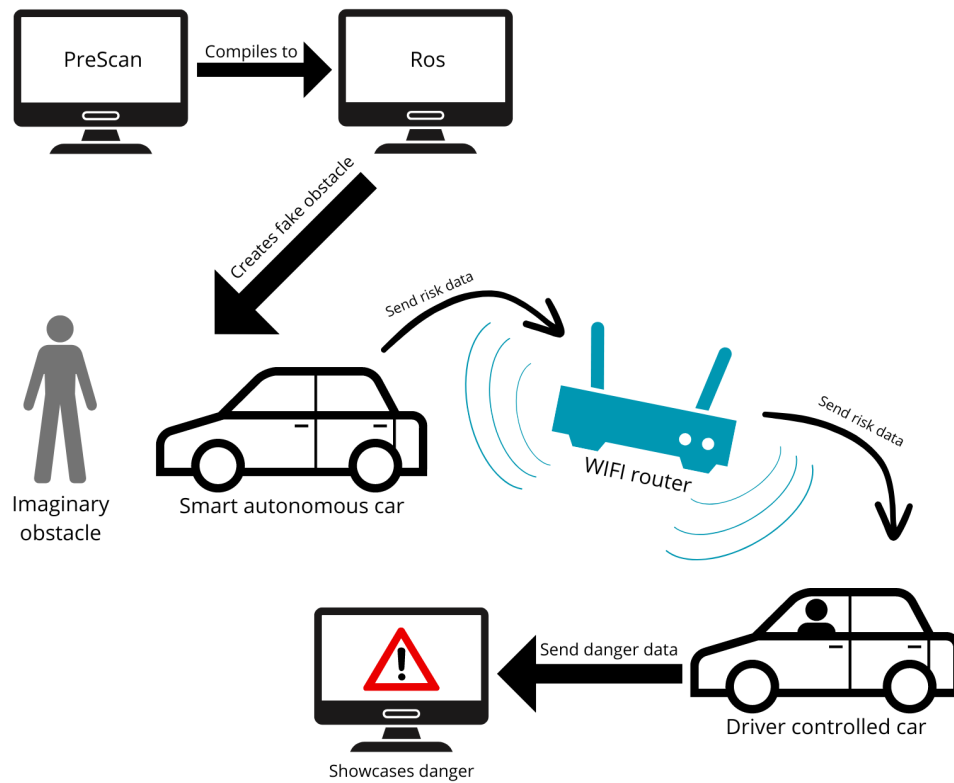
The complexity analysis details of the kd-tree operations are provided in [7]

- Building a kd-tree: $O(n \log n)$
- Nearest neighbor search: $O(\log n)$
- m nearest neighbors: $O(m \log n)$

These operations will serve as the basis of our segmentation, in which each of the LIDAR scans is broken down into its constituent laser scans; they will, in turn, aid us in using thresholding techniques that are specifically adapted to separate ground points from those that are non-ground.

4.2 Expand our project

As mentioned before we did not achieve all of the tasks necessary to complete our problem statement. We want to continue by making the risk field something that is communicated between vehicles. Right now it is one vehicle assessing its risk but we need to have two vehicles where the assessed risk is communicated between them. We are also going to integrate our project with PreScan, this will allow us to dynamically test the interface we have created. The figure below showcases what the total structure of our project should become when we have completed it.



When we have achieved this structure we want to work towards a research paper that showcases this potential new tool for increasing road awareness.

5 Use of AI tools

GPT has been used in this document for reformatting the scalar field math equations in latex.

References

- [1] Raghvendra V. Cowlagi, Rebecca C. Debski, and Alexander M. Wyglinski. Risk quantification for automated driving using information from v2v basic safety messages. In *2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*, pages 1–5, 2021.
- [2] Matviienko A. Berger M. Pfleging B. Martens-M. H. Terken J. M. B. Dey, D. Communicating the intention of an automated vehicle to pedestrians: The contributions of ehmi and vehicle behavior. *IT-Information Technology*, 63(2), pages 123–141, 2021.
- [3] Guus Engels, Nerea Aranjuelo, Ignacio Arganda-Carreras, Marcos Nieto, and Oihana Otaegui. 3d object detection from lidar data using distance dependent feature extraction. pages 289–300, 01 2020.
- [4] ARASH KHABBAZ SABERI JAN-PIETER PAARDEKOOPEL OLAF OP DEN CAMP ERWIN DE GELDER, HALA ELROFAI1 and BART DE SCHUTTER. Risk quantification for automated driving systems in real-world driving scenarios. *IEEE access*, 2021.
- [5] Shuaixin Liu, JianYing Zheng, Xiang Wang, ZhenYao Zhang, and Rongchuan Sun. Target detection from 3d point-cloud using gaussian function and cnn. pages 562–567, 06 2019.
- [6] Ao Luo, Linxin Song, Keisuke Nonaka, Kyohei Unno, Heming Sun, Masayuki Goto, and Jiro Katto. Scp: Spherical-coordinate-based learned point cloud compression. 08 2023.
- [7] Zhongzhen Luo, Saeid R. Habibi, and Martin von Mohrenschildt. Lidar based real time multiple vehicle detection and tracking. *World Academy of Science, Engineering and Technology, International Journal of Transport and Vehicle Engineering*, 3, 2016.
- [8] Sumbal Malik Muhammad Talha Zia Najla Alkaabi Jalal Khan Manzoor Ahmed Khan, Hesham El Sayed. A journey towards fully autonomous driving - fueled by a smart communication system. *Vehicular Communications*, 2022.

A Individual contributions

A.1 Nora

In our team, we decided not to assign roles to enable everyone to show their strengths. Despite this structure, I quickly took on an organizational role. Organizing meetings and other required tasks became my unwritten responsibility. As I like to have a clear overview of where everybody stands, I enjoy this role and will continue supplying my organizational skills next year.

Regarding the responsibilities within the technical part of our project, I took up a more back-end role. This was mainly because working with physical prototypes and communicating with hardware was something fairly new to me, while my team members both had previous experience.

One of the main things that became mostly my responsibility was PreScan. This is a tool that can create artificial obstacles which the vehicles can then 'see' and assess their risk based on. Unfortunately, we were unable to integrate this with our deliverables for this year, but next year PreScan will be a very useful tool.

Another thing I spent a lot of time on was optimizing the program, as the initial speed was not fast enough to do real-time risk assessment. We tried a lot of different approaches when optimizing the program, pooling, multiprocessing, etc., but none of them had a significant effect until Daniel tried compiling with Numba.

Most of my time, however, was spent learning and experimenting with the cars in the lab. This is not something that directly produced deliverables in our project, but I would never have been able to achieve the results that we did had I not spent enough time learning.

Another thing I helped with was finding and analyzing relevant research papers. The majority of the first few months went towards researching and defining a proper problem statement. I contributed here by reading a lot of research papers and summarizing the key points for my group.

I also took charge of creating the presentations for all the moments we needed to showcase our progress. This does not directly affect our deliverables but was still needed.

Lastly, I contributed to the initial use of the LIDAR sensor. In one of our early lab sessions, I spent a significant amount of time understanding the LIDAR sensor and its data. In later lab sessions, the responsibility of working with the sensor was passed on, but I had to support this switch by explaining the sensor and its data.

Overall, our project was very collaborative, meaning that everybody helped out everywhere. There was nothing that was someone's sole responsibility. This worked well as it meant that if something went wrong, everybody was also responsible.

A.2 Ismail

My main contribution to the team's work was researching, developing, and implementing the mathematical framework for our risk quantification model. This involved formulating the equations and algorithms that form the foundation of the risk field, as well as ensuring their accuracy and effectiveness. Additionally, I dedicated a significant portion of my effort to visualizing the risk field, which was crucial for understanding and demonstrating the model's impact.

Alongside Nora, I contributed to the literature review that served as the backbone of our risk field research. This involved analyzing existing studies and theories to provide a comprehensive background for our work.

Moreover, I played a key role in the technical setup and integration of our project components. I was responsible for configuring the RC car, which involved coding and learning in ROS. Furthermore, I established a network server connection link between the car and our PC, which enabled real-time data transmission and provided us with a live overview of the data allowing for live calculation and rendering of the risk field. In summary, my contributions encompassed several critical areas:

1. **Risk Field:** Developed and implemented the mathematical framework for risk quantification and visualization.
2. **Server Communication:** Created a WiFi link for real-time data transmission between the RC car and our PC.
3. **ROS Setup and Car Coding:** Configured and coded the ROS for the RC car, ensuring effective autonomous operation.
4. **Literature Research:** Collaborated on the literature review, providing a strong theoretical foundation for our research.

A.3 Daniel

We took the flat hierarchy approach, so each member was an engineer without predetermined roles. It kept us malleable and reactive to what the project needed. My role within that aspect of teamwork was taking on responsibilities where I could work in different areas while focusing mostly on integrating and optimizing the LIDAR system. With a flat structure, there was an assurance of a dynamic and collaborative environment, which gave room for any member to exploit their strengths productively and contribute to different project aspects.

As an example, I have continued to do further serious research work on a LIDAR-based detection system and practically implement it back using the ROS. This is in an attempt to properly coordinate the communication between major components such as the LIDAR sensor through ROS. Subsequently, I had to understand the framework of ROS, experiment with different configurations, and finally implement a configurable and robust system appropriate for real-time data but that also dealt with such data efficiently.

In practice, my interaction with ROS has been quite detailed; I have run many experiments, fine-tuning them towards the communication of a LIDAR sensor with other subsystems. I also wrote some custom ROS nodes to process and publish LIDAR data to ensure that the described system is robust and reliable under all working conditions.

I also played an assisting role in harnessing the power of Prescan and Simulink for use in the simulation of the environment to be tested. This helped make sure we were achieving realistic environments for simulating our scenarios and then testing them in various conditions. This involved setting up simulation models serving as replicas of real-world situations to help us validate system validity before actual deployment.

This project was characterized by a significant amount of collaboration, in that I had to closely engage with another student in the project during the design and implementation of code, and then proceed further to conduct tests on the mathematical algorithm of Risk Assessment. We worked by examining a referenced paper on risk field algorithms; I helped implement and subsequently optimized the algorithm to bring in an improvement in execution time from 3 seconds to less than 0.1 seconds. This optimization was achieved when recoding into Numba for practical real-time application to increase the overall performance of our system.

In addition to that, I consistently did general work in the lab and did whatever was thrown my way to help out the rest of the team and make our project run successfully. This also included solving hardware issues, optimizing software for working, and arranging effective and efficient communication between group members.

I engaged in reading the research papers and analyzing them which has been a significant help for the development of our project and planning into the future. Keeping up with all the latest research papers in LIDAR technology and related technological areas was important for strategic decisions regarding the direction of the research and development efforts.

Through my engagement, our team has managed to accomplish the successful realization of a fully functional and efficient LIDAR-based functionality detection and risk assessment system. My work with ROS and integration of LIDAR, including the optimization of the algorithms, was of great importance in realizing the objectives of the project. Moreover, my involvement in setting up the simulation and research extensively supported laying the groundwork for future advancements and refinements in our project. I am beyond excited to continue with this project next year, and work with my team in hopefully bringing this project outside the Lab.

B Code

B.1 Final Risk Field Simulation Code

```
1 import numpy as np
2 import time
3 import cProfile
4 import pstats
5 from numba import njit, prange
6 from vector import *
7 from gui import *
8
9 # Constants
10 v_o = 24.2 # m/s
11 epsilon0 = 0.1
12 epsilon1 = 1 / (-np.log(epsilon0**2))*0.5
13 epsilon2 = 0.2420
14 epsilon3 = 5
15 epsilon4 = 0.05
16 epsilon5 = 0.0001
17 epsilon6 = 100
18
19 # Calculation functions
20 @njit(fastmath=True)
21 def calc_d_i(v_o, V_i):
22     return np.array([3 * (v_o + np.abs(V_i[0])), 3 * (v_o + np.abs(V_i[1]))])
23
24 @njit(fastmath=True)
25 def calc_beta_i(d_i, v_o, V_i, epsilon2, epsilon3, epsilon4):
26     return np.array([np.log(d_i[0] * (v_o - np.abs(V_i[0]) - epsilon2) / (2 * (np.abs(V_i[0] +
27         epsilon2))))),
28         np.log(d_i[1] * (epsilon3 - np.abs(V_i[1]) - epsilon4) / (2 * (np.abs(V_i[1]
29         + epsilon4))))])
30
31 @njit(fastmath=True)
32 def calc_alpha_i(V_i, v_o, epsilon1, epsilon2, epsilon3, epsilon4):
33     return np.array([epsilon1 * np.log((v_o + np.abs(V_i[0]) + epsilon2) / (v_o - np.abs(V_i[0]) -
34         epsilon2))),
35         epsilon1 * np.log((epsilon3 + np.abs(V_i[1]) + epsilon4) / (epsilon3 - np.abs
36         (V_i[1]) - epsilon4))])
37
38 @njit(fastmath=True)
39 def calc_delta_i(V_i, P_i, P, exp_beta_i):
40     return np.array([np.sign(V_i[0]) * (P[0] - P_i[0]) + exp_beta_i[0],
41         np.sign(V_i[1]) * (P[1] - P_i[1]) + exp_beta_i[1]])
42
43 @njit(fastmath=True)
44 def f_i(delta, alpha, beta):
45     return np.exp(-0.5 * ((np.log(delta[0]) - beta[0]) / alpha[0])**2) * np.exp(-0.5 * ((np.log(
46         delta[1]) - beta[1]) / alpha[1])**2)
47
48 @njit(parallel=True, fastmath=True)
49 def calculate_grid(grid, v_o, V_l, P_l, epsilon2, epsilon3, epsilon4, epsilon5, epsilon6):
50     d_l = calc_d_i(v_o, V_l)
51     beta_l = calc_beta_i(d_l, v_o, V_l, epsilon2, epsilon3, epsilon4)
52     exp_beta_i = np.exp(beta_l)
53     alpha_l = calc_alpha_i(V_l, v_o, epsilon1, epsilon2, epsilon3, epsilon4)
54
55     for x in prange(300):
56         for y in range(300):
57             delta_l = calc_delta_i(V_l, P_l, np.array([x, y]), exp_beta_i)
58             grid[y][x] = epsilon6 * f_i(delta_l, alpha_l, beta_l) if (delta_l[0] > 0 and delta_l
59                 [1] > 0) else epsilon5
60
61 calc_time = []
62 disp_time = []
63 disp_time_total = []
64
65 def main():
66     for x_moving in range(40, 70, 2):
67         P_l = np.array([x_moving, 60])
```

```

62     V_1 = np.array([6, 0.1])
63
64     grid_size = 300
65     grid = np.zeros((grid_size, grid_size))
66
67     t1 = time.time()
68     calculate_grid(grid, v_o, V_1, P_1, epsilon2, epsilon3, epsilon4, epsilon5, epsilon6)
69     t2 = time.time()
70     calc_time.append(t2 - t1)
71
72     t3 = time.time()
73     show_grid(grid)
74     t4 = time.time()
75     disp_time.append(t4 - t3)
76     print(f"Display update time: {np.mean(disp_time)}, calculation time: {np.mean(calc_time)}")
77     disp_time_total.append(np.mean(disp_time))
78
79 if __name__ == '__main__':
80     cProfile.run('main()', 'profiling_results')
81
82 p = pstats.Stats('profiling_results')
83 p.strip_dirs().sort_stats('cumtime').print_stats(100)

```